



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

INDEX

IMPORTS	3
Data	3
KNN Classifier	4
Standardization	6
Cross Validation.....	7
Main Program.....	11
Plot Results	11
Analysis Results	12
Conclusion	13
KNN WTH SKLEARN	14

IMPORTS

```
from scipy.io import loadmat
```

برای خواندن و بارگذاری فایل‌های MATLAB با پسوند .mat در پایتون استفاده می‌شود.

```
import numpy as np
```

کتابخانه NumPy را برای انجام محاسبات عددی، کار با آرایه‌ها و عملیات ریاضی وارد می‌کند.

```
import matplotlib.pyplot as plt
```

کتابخانه Matplotlib را برای رسم نمودارها و نمایش گرافیکی داده‌ها وارد می‌کند.

DATA

```
mat = loadmat('Thyroid.mat')
```

فایل MATLAB به نام Thyroid.mat را خوانده و محتویات آن را به صورت یک دیکشنری در متغیر mat ذخیره می‌کند.

```
Data = mat['Data']
```

متغیر Data موجود در فایل MATLAB را استخراج کرده و در متغیر Data برای استفاده در برنامه ذخیره می‌کند.

KNN Classifier

این بخش کد پیاده‌سازی ساده‌ای از الگوریتم KNN¹ است. ایده اصلی KNN این است که برای پیش‌بینی برچسب یک نمونه جدید، ابتدا فاصله آن نمونه را با تمام داده‌های آموزشی محاسبه می‌کنیم، سپس نزدیک‌ترین همسایه‌ها را پیدا می‌کنیم و در نهایت بر اساس رأی اکثریت آن همسایه‌ها، کلاس نمونه جدید را تعیین می‌کنیم.

```
class KNN:
```

در این بخش، کلاس KNN تعریف شده است. این کلاس تمام قابلیت‌های مورد نیاز برای آموزش و پیش‌بینی را در خود نگه می‌دارد.

```
def __init__(self, k):  
    self.k = k
```

زمانی که یک شیء از این کلاس ساخته می‌شود، متد `__init__` اجرا می‌شود. این متد یک پارامتر به نام `k` دریافت می‌کند که تعداد همسایه‌های نزدیک مورد استفاده در تصمیم‌گیری را مشخص می‌کند. دستور `self.k = k` این مقدار را در شیء ذخیره می‌کند تا بعداً در مراحل پیش‌بینی قابل استفاده باشد.

```
def fit(self, X, y):  
    self.X_train = X  
    self.y_train = y
```

در بسیاری از الگوریتم‌های یادگیری ماشین، متد `fit` وظیفه آموزش مدل را برعهده دارد؛ اما در KNN عملاً فرآیند آموزش پیچیده‌ای وجود ندارد. این الگوریتم تنها داده‌های آموزشی را در حافظه نگه می‌دارد. به همین دلیل در این متد، آرایه ویژگی‌ها (`X`) در متغیر `self.X_train` و برچسب‌ها (`y`) در متغیر `self.y_train` ذخیره می‌شوند. در واقع مدل فقط داده‌ها را به خاطر می‌سپارد تا هنگام پیش‌بینی بتواند آن‌ها را با نمونه‌های جدید مقایسه کند.

```
def euclidean_distance(self, x1, x2):  
    return np.sqrt(np.sum((x1 - x2) ** 2))
```

متد `euclidean_distance` فاصله اقلیدسی بین دو نقطه را محاسبه می‌کند. این متد دو بردار `x1` و `x2` را دریافت می‌کند. عبارت $(x1 - x2)$

اختلاف هر ویژگی را محاسبه می‌کند. سپس این اختلاف‌ها به توان دو می‌رسند تا مقادیر منفی از بین بروند. تابع `np.sum` مجموع این مربع‌ها را

محاسبه می‌کند و در نهایت `np.sqrt` از آن جذر می‌گیرد. نتیجه همان فرمول معروف فاصله اقلیدسی است:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

هر چه این مقدار کوچک‌تر باشد، دو نمونه به یکدیگر شبیه‌تر هستند.

```
def predict_single(self, sample):
```

مهم‌ترین بخش الگوریتم در متد `predict_single` قرار دارد. این متد وظیفه پیش‌بینی کلاس یک نمونه را برعهده دارد. ورودی آن یک نمونه جدید به نام `sample` است.

```
distances = []
```

در ابتدای متد، یک لیست خالی به نام `distances` ساخته می‌شود تا فاصله نمونه جدید با تمام نمونه‌های آموزشی در آن ذخیره شود.

```
for train_sample in self.X_train:  
    distance = self.euclidean_distance(sample, train_sample)  
    distances.append(distance)
```

سپس یک حلقه `for` روی تمام داده‌های آموزشی اجرا می‌شود. در هر تکرار، یکی از نمونه‌های آموزشی در متغیر `train_sample` قرار می‌گیرد. فاصله بین نمونه جدید و این نمونه آموزشی با استفاده از متد `euclidean_distance` محاسبه می‌شود. مقدار فاصله در متغیر

¹ K-Nearest Neighbors

distance ذخیره شده و به لیست distances اضافه می‌شود. پس از پایان حلقه، این لیست شامل فاصله نمونه جدید تا تک تک نمونه‌های آموزشی خواهد بود.

```
distances = np.array(distances)
```

در ادامه، لیست فاصله‌ها با دستور np.array(distances) به آرایه NumPy تبدیل می‌شود. این کار باعث می‌شود بتوان از توابع NumPy برای پردازش سریع‌تر استفاده کرد.

```
k_indices = np.argsort(distances)[:self.k]
```

سپس با دستور np.argsort(distances) اندیس‌های فاصله‌ها بر اساس ترتیب صعودی مرتب می‌شوند. یعنی اندیس نزدیک‌ترین نمونه در ابتدای آرایه قرار می‌گیرد. عبارت [:self.k] فقط اولین k اندیس را انتخاب می‌کند. بنابراین متغیر k_indices شامل اندیس نزدیک‌ترین همسایه‌ها خواهد بود.

```
k_labels = self.y_train[k_indices]
```

در این مرحله، با استفاده از این اندیس‌ها لیبل‌های همسایه‌های نزدیک استخراج می‌شوند. اگر مثلاً نزدیک‌ترین همسایه‌ها کلاس‌های [0, 1, 1] داشته باشند، همین مقادیر در k_labels ذخیره می‌شود.

```
unique_labels, counts = np.unique(k_labels, return_counts=True)
```

اکنون باید مشخص شود کدام کلاس بیشترین تکرار را در میان همسایه‌ها دارد. برای این کار از تابع np.unique استفاده شده است. این تابع دو خروجی برمی‌گرداند: اول لیست کلاس‌های یکتا و دوم تعداد تکرار هر کلاس. برای مثال اگر k_labels برابر [1, 1, 0] باشد، خروجی به صورت unique_labels = [0, 1] و counts = [1, 2] خواهد بود.

```
prediction = unique_labels[np.argmax(counts)]
```

تابع np.argmax(counts) اندیس بزرگ‌ترین مقدار موجود در counts را پیدا می‌کند. در مثال بالا، بزرگ‌ترین مقدار ۲ است که اندیس آن ۱ می‌باشد. بنابراین کلاس موجود در unique_labels[1] یعنی کلاس ۱ انتخاب می‌شود. این همان مرحله رأی‌گیری اکثریت یا Majority Voting است.

```
return prediction
```

در نهایت متد predict_single برچسب پیش‌بینی شده را با دستور return prediction بازمی‌گرداند.

```
def predict(self, X):
```

متد آخر یعنی predict برای پیش‌بینی چندین نمونه به صورت هم‌زمان استفاده می‌شود. ورودی این متد آرایه‌ای از نمونه‌ها به نام X است..

```
predictions = []
```

ابتدا یک لیست خالی به نام predictions ایجاد می‌شود

```
for sample in X:
```

سپس روی تمام نمونه‌های موجود در X حلقه اجرا می‌شود

```
predictions.append(self.predict_single(sample))
```

برای هر نمونه، متد predict_single فراخوانی می‌شود و نتیجه به لیست predictions اضافه می‌گردد

```
return np.array(predictions)
```

پس از اتمام حلقه، تمام پیش‌بینی‌ها در قالب یک آرایه NumPy برگردانده می‌شوند. بنابراین جریان کلی اجرای برنامه به این شکل است: ابتدا با fit داده‌های آموزشی ذخیره می‌شوند. سپس هنگام فراخوانی predict، هر نمونه جدید به predict_single ارسال می‌شود. برای آن نمونه فاصله تا همه داده‌های آموزشی محاسبه می‌شود، نزدیک‌ترین k همسایه پیدا می‌شوند، رأی اکثریت بین آن‌ها گرفته می‌شود و کلاس نهایی به عنوان خروجی برگردانده می‌شود. این دقیقاً منطق اصلی الگوریتم KNN است که به آن «یادگیری مبتنی بر نمونه» یا Lazy Learning نیز گفته می‌شود، زیرا آموزش واقعی در زمان پیش‌بینی انجام می‌گیرد، نه در زمان فراخوانی fit

این بخش از کد وظیفه استانداردسازی داده‌ها را برعهده دارد. استانداردسازی یکی از مراحل مهم پیش‌پردازش داده‌ها در یادگیری ماشین است، به‌خصوص در الگوریتم‌هایی مانند KNN که بر پایه محاسبه فاصله کار می‌کنند. اگر مقیاس ویژگی‌ها با یکدیگر متفاوت باشد، ویژگی‌هایی که مقادیر بزرگ‌تری دارند تأثیر بیشتری بر فاصله خواهند گذاشت و نتیجه مدل ممکن است دچار خطا شود. به همین دلیل قبل از آموزش مدل، داده‌ها معمولاً استانداردسازی می‌شوند.

```
def standardize(train_X, test_X):
```

این تابع دو ورودی دریافت می‌کند. train_X شامل داده‌های آموزشی و test_X شامل داده‌های آزمون است.

```
mean = np.mean(train_X, axis=0)
```

تابع `np.mean` میانگین را محاسبه می‌کند. پارامتر `axis=0` به این معناست که میانگین برای هر ستون به صورت جداگانه محاسبه شود. اگر هر ستون نماینده یک ویژگی باشد، نتیجه آرایه‌ای خواهد بود که میانگین هر ویژگی را در خود نگه می‌دارد.

```
std = np.std(train_X, axis=0)
```

تابع `np.std` میزان پراکندگی داده‌ها را اندازه‌گیری می‌کند. همانند میانگین، پارامتر `axis=0` باعث می‌شود انحراف معیار برای هر ستون به صورت مستقل محاسبه شود. نتیجه آرایه‌ای است که برای هر ویژگی یک مقدار انحراف معیار در خود دارد.

```
std[std == 0] = 1
```

گاهی ممکن است تمام مقادیر یک ویژگی دقیقاً یکسان باشند. در چنین حالتی انحراف معیار آن ویژگی برابر صفر خواهد شد. اگر در مرحله استانداردسازی بخواهیم بر صفر تقسیم کنیم، برنامه با خطا مواجه می‌شود. این خط تمام مقادیر صفر موجود در آرایه `std` را به ۱ تغییر می‌دهد تا از بروز خطای تقسیم بر صفر جلوگیری شود.

```
train_scaled = (train_X - mean) / std
```

```
test_scaled = (test_X - mean) / std
```

این فرمول استانداردسازی را نشان می‌دهد. در این فرمول، از هر مقدار ویژگی میانگین آن ویژگی کم می‌شود و سپس نتیجه بر انحراف معیار تقسیم می‌شود. حاصل این عملیات داده‌ای است که میانگین آن تقریباً صفر و انحراف معیار آن تقریباً یک خواهد بود.

نکته بسیار مهم این است که برای استانداردسازی داده‌های آزمون نیز از همان میانگین و انحراف معیار محاسبه‌شده از داده‌های آموزشی استفاده می‌شود. دلیل این کار جلوگیری از نشت اطلاعات^۲ است. اگر میانگین و انحراف معیار داده‌های آزمون را جداگانه محاسبه کنیم، مدل به اطلاعاتی دسترسی پیدا می‌کند که در زمان آموزش نباید از آن‌ها اطلاع داشته باشد.

```
return train_scaled, test_scaled
```

بنابراین خروجی تابع شامل دو آرایه است؛ یکی داده‌های آموزشی استاندارد شده و دیگری داده‌های آزمون استاندارد شده.

^۲ Data Leakage

این تابع برای ایجاد Fold ها در Cross Validation استفاده می‌شود. در بسیاری از مسائل یادگیری ماشین، برای ارزیابی بهتر عملکرد مدل، داده‌ها به چند بخش تقسیم می‌شوند و مدل چندین بار آموزش و آزمایش می‌شود. هر یک از این بخش‌ها را یک Fold می‌نامند. هدف این روش این است که مدل روی قسمت‌های مختلف داده آزمایش شود و ارزیابی دقیق‌تر و قابل اعتمادتری به دست آید.

```
def create_folds(data, n_folds=5):
```

این تابع دو ورودی دریافت می‌کند. ورودی اول `data` است که شامل کل داده‌ها می‌شود و ورودی دوم `n_folds` است که تعداد Fold های موردنظر را مشخص می‌کند. مقدار پیش فرض این پارامتر برابر ۵ در نظر گرفته شده است

```
np.random.seed(42)
```

تابع `seed` باعث می‌شود عملیات تصادفی در هر بار اجرای برنامه نتیجه یکسانی تولید کند. به عبارت دیگر، هر بار که این تابع اجرا شود، ترتیب داده‌های درهم‌ریخته شده دقیقاً مشابه خواهد بود. این کار برای تکرارپذیری آزمایش‌ها اهمیت زیادی دارد؛ زیرا توسعه‌دهندگان می‌توانند نتایج یکسانی را دوباره تولید کنند.

```
shuffled_data = data.copy()
```

هدف از این کار جلوگیری از تغییر داده‌های اصلی است. اگر مستقیماً روی `data` عملیات انجام شود، ترتیب داده‌های اولیه از بین می‌رود. با ایجاد یک نسخه کپی، داده‌های اصلی دست‌نخورده باقی می‌مانند و تمام تغییرات روی نسخه جدید اعمال می‌شود.

```
np.random.shuffle(shuffled_data)
```

تابع `shuffle` ترتیب سطرها را به صورت تصادفی تغییر می‌دهد. این مرحله بسیار مهم است؛ زیرا اگر داده‌ها به ترتیب خاصی ذخیره شده باشند (مثلاً تمام نمونه‌های یک کلاس پشت سر هم قرار گرفته باشند)، تقسیم مستقیم آن‌ها به Fold ها می‌تواند باعث ایجاد Fold های نامتعادل شود. درهم‌ریختن داده‌ها باعث می‌شود نمونه‌ها به شکل یکنواخت‌تری در Fold های مختلف توزیع شوند.

```

folds = np.array_split(
    shuffled_data,
    n_folds)

```

تابع `np.array_split` آرایه را به تعداد مشخصی قسمت تقسیم می‌کند. در اینجا کل داده‌های درهم‌ریخته شده به تعداد `n_folds` بخش تقسیم می‌شوند. اگر تعداد نمونه‌ها دقیقاً بر تعداد Fold ها بخش‌پذیر نباشد، NumPy به صورت خودکار اختلاف را مدیریت می‌کند و بعضی Fold ها ممکن است یک نمونه بیشتر از بقیه داشته باشند.

برای مثال اگر ۱۰۰ نمونه داشته باشیم و بخواهیم آن‌ها را به ۵ Fold تقسیم کنیم، هر Fold شامل ۲۰ نمونه خواهد بود. اما اگر ۱۰۳ نمونه داشته باشیم، بعضی Fold ها ۲۱ نمونه و بعضی دیگر ۲۰ نمونه خواهند داشت.

نتیجه این تقسیم‌بندی در متغیر `folds` ذخیره می‌شود. این متغیر در واقع یک لیست از آرایه‌ها است که هر آرایه نماینده یک Fold است.

```
return folds
```

بنابراین خروجی تابع مجموعه‌ای از Fold ها است که می‌توان از آن‌ها در فرآیند Cross Validation استفاده کرد.

🚩 تابع cross_validation_knn

این تابع قلب اصلی آزمایش است و وظیفه دارد الگوریتم KNN را با استفاده از 5-Fold Cross Validation ارزیابی کند. همچنین این امکان را فراهم می‌کند که عملکرد مدل هم در حالت عادی و هم در حالت استانداردسازی شده داده‌ها بررسی شود. در نهایت، میانگین خطای طبقه‌بندی برای مقادیر مختلف K محاسبه و گزارش می‌شود.

```
def cross_validation_knn(data, normalize=False):
```

این تابع دو ورودی دریافت می‌کند. ورودی اول data است که شامل کل داده‌های مسئله می‌شود. ورودی دوم normalize یک متغیر بولی است که مشخص می‌کند آیا قبل از آموزش مدل باید داده‌ها استانداردسازی شوند یا خیر. مقدار پیش فرض آن False است؛ بنابراین اگر کاربر چیزی مشخص نکند، داده‌ها بدون استانداردسازی استفاده خواهند شد.

```
    folds = create_folds(data, n_folds=5)
```

در اولین مرحله، داده‌ها به Fold های موردنیاز تقسیم می‌شوند. تابع create_folds که قبلاً تعریف شده بود، داده‌ها را به پنج بخش تقسیم می‌کند. این پنج بخش در فرآیند Cross Validation مورد استفاده قرار خواهند گرفت.

```
    k_values = range(1, 11)
```

سپس محدوده مقادیر K مشخص می‌شود. این دستور مجموعه‌ای از مقادیر ۱ تا ۱۰ را تولید می‌کند. هدف این است که عملکرد KNN برای K های مختلف بررسی شود تا مشخص شود کدام مقدار بهترین نتیجه را ایجاد می‌کند.

```
    average_errors = []
```

در ادامه، چند خط برای نمایش عنوان خروجی چاپ می‌شوند.

```
    for k in k_values:
```

در هر تکرار، یک مقدار جدید برای K انتخاب می‌شود و کل فرآیند اعتبارسنجی متقابل برای آن اجرا می‌گردد.

```
        fold_errors = []
```

برای ذخیره خطاهای Fold های مختلف یک لیست خالی ساخته می‌شود. سپس حلقه دوم آغاز می‌شود.

```
        for i in range(5):
```

این حلقه پنج بار اجرا می‌شود؛ زیرا پنج Fold داریم. در هر بار اجرا یکی از Fold ها نقش داده اعتبارسنجی را بازی می‌کند و چهار Fold دیگر برای آموزش استفاده می‌شوند.

```
            validation_set = folds[i]
```

ابتدا Fold اعتبارسنجی انتخاب می‌شود. به عنوان مثال اگر $i = 2$ باشد، Fold سوم به عنوان مجموعه اعتبارسنجی انتخاب خواهد شد.

```
            train_folds = [  
                folds[j]  
                for j in range(5)
```



```
if j != i]
```

سپس Fold های آموزشی مشخص می شوند: این عبارت یک List Comprehension است. تمام Fold ها به جز Fold فعلی انتخاب می شوند. بنابراین اگر Fold سوم برای اعتبارسنجی استفاده شود، چهار Fold دیگر به عنوان داده آموزشی در نظر گرفته می شوند.

```
train_set = np.vstack(train_folds)
```

در مرحله بعد، این Fold های آموزشی به یک مجموعه واحد تبدیل می شوند. تابع `vstack` تمام آرایه ها را به صورت عمودی روی هم قرار می دهد. در نتیجه یک مجموعه آموزشی کامل ایجاد می شود.

```
X_train = train_set[:, :-1]
y_train = train_set[:, -1]

X_val = validation_set[:, :-1]
y_val = validation_set[:, -1]
```

حالا ویژگی ها و برچسب ها از داده های آموزشی جدا می شوند. همین کار برای داده های اعتبارسنجی نیز انجام می شود. اکنون ویژگی ها و برچسب های مجموعه اعتبارسنجی آماده هستند.

```
if normalize:
```

اگر این شرط برقرار باشد، داده های آموزشی و اعتبارسنجی استانداردسازی می شوند

```
X_train, X_val = standardize(
    X_train,
    X_val)
```

همان طور که در تابع استانداردسازی دیدیم، میانگین و انحراف معیار فقط از داده های آموزشی محاسبه می شود و سپس روی هر دو مجموعه اعمال می شود تا از نشت اطلاعات جلوگیری گردد.

```
model = KNN(k=k)
```

اکنون مدل KNN ساخته می شود.

```
model.fit(
    X_train,
    y_train)
```

پس از ساخت مدل، داده های آموزشی به آن داده می شوند. در KNN این مرحله تنها باعث ذخیره شدن داده های آموزشی در حافظه می شود. در ادامه، پیش بینی روی داده های اعتبارسنجی انجام می شود.

```
predictions = model.predict(X_val)
```

مدل برای هر نمونه موجود در مجموعه اعتبارسنجی نزدیکترین همسایه‌ها را پیدا می‌کند و کلاس آن نمونه را پیش‌بینی می‌کند.

```
error = np.mean(
    predictions != y_val)
```

ابتدا عبارت `predictions != y_val` بررسی می‌کند که کدام پیش‌بینی‌ها اشتباه هستند. خروجی این مقایسه یک آرایه بولی شامل مقادیر True و False خواهد بود.

```
fold_errors.append(error)
```

بعد از پایان حلقه پنج‌تایی، پنج مقدار خطا در `fold_errors` وجود دارد که هر کدام مربوط به یک Fold هستند.

```
avg_error = np.mean(fold_errors)
average_errors.append(avg_error)
```

اکنون میانگین این خطاها محاسبه می‌شود. این مقدار نشان‌دهنده عملکرد کلی مدل برای مقدار فعلی K است. سپس این مقدار در لیست نهایی ذخیره می‌شود.

```
return list(k_values), average_errors
```

پس از آنکه تمام مقادیر K از ۱ تا ۱۰ بررسی شدند، تابع به انتهای خود می‌رسد و نتایج را بازمی‌گرداند: در خروجی، لیستی از مقادیر K و لیستی از میانگین خطاهای متناظر با آن‌ها برگردانده می‌شود.

به طور کلی، این تابع ابتدا داده‌ها را به پنج Fold تقسیم می‌کند، سپس برای هر مقدار K بین ۱ تا ۱۰، فرآیند ۵-Fold Cross Validation را اجرا می‌کند. در هر تکرار یک Fold برای اعتبارسنجی و چهار Fold برای آموزش استفاده می‌شوند. در صورت نیاز داده‌ها استانداردسازی می‌شوند، مدل KNN آموزش می‌بیند، پیش‌بینی انجام می‌شود و خطای طبقه‌بندی محاسبه می‌گردد. در نهایت میانگین خطای هر مقدار K به دست می‌آید و برای مقایسه عملکرد مدل در مقادیر مختلف K مورد استفاده قرار می‌گیرد. این دقیقاً روشی است که معمولاً برای انتخاب بهترین مقدار K در الگوریتم KNN به کار می‌رود.

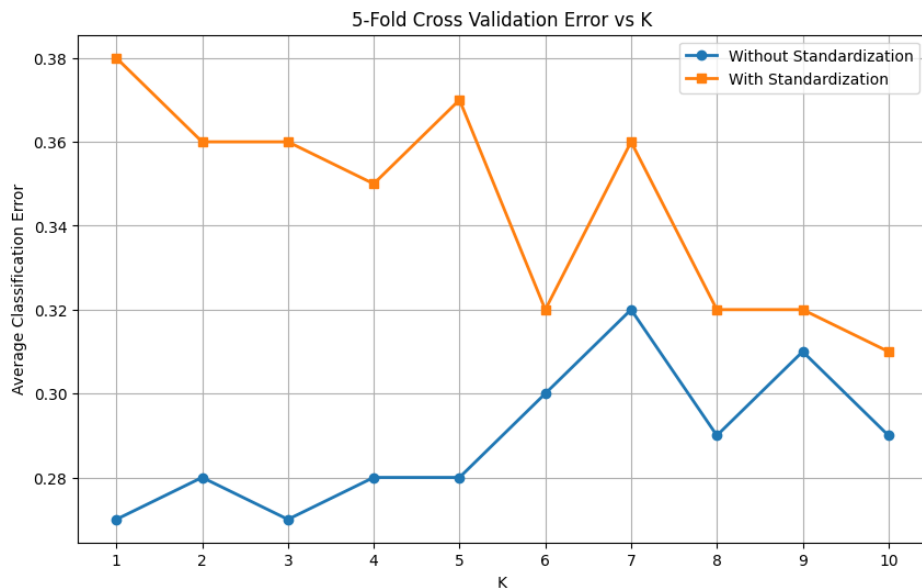
Main Program

این بخش، قسمت نهایی برنامه است و وظیفه اجرای آزمایش‌ها و نمایش نتایج را بر عهده دارد. در ابتدا چند خط برای زیباسازی خروجی چاپ می‌شود و عنوان طبقه‌بندی بیماری تیروئید با استفاده از KNN نمایش داده می‌شود. سپس تابع `cross_validation_knn` دو بار فراخوانی می‌شود. در اجرای اول، مقدار `normalize=False` قرار داده شده است؛ بنابراین الگوریتم KNN روی داده‌های خام و بدون استانداردسازی اجرا می‌شود. در اجرای دوم، مقدار `normalize=True` است و داده‌ها قبل از آموزش و ارزیابی استانداردسازی می‌شوند. خروجی هر بار اجرا شامل لیست مقادیر K و میانگین خطای طبقه‌بندی متناظر با هر K است.

پس از اتمام هر دو آزمایش، بهترین مقدار K برای هر حالت تعیین می‌شود. تابع `np.argmin` اندیس کمترین خطا را پیدا می‌کند و از آن برای استخراج مقدار K متناظر استفاده می‌شود. همچنین با `np.min` کمترین مقدار خطا محاسبه می‌شود. در نهایت، نتایج در قالب یک گزارش نهایی چاپ می‌شوند و بهترین مقدار K به همراه کمترین خطای طبقه‌بندی برای هر دو حالت «با استانداردسازی» و «بدون استانداردسازی» نمایش داده می‌شود. این مقایسه نشان می‌دهد که استانداردسازی داده‌ها چه تأثیری بر عملکرد الگوریتم KNN داشته و کدام مقدار K بهترین نتیجه را برای مسئله تشخیص بیماری تیروئید ایجاد کرده است.

Plot Results

این بخش از برنامه نتایج به دست آمده از اعتبارسنجی متقابل را به صورت نمودار رسم می‌کند تا مقایسه عملکرد KNN در مقادیر مختلف K آسان‌تر شود. دو منحنی روی نمودار نمایش داده می‌شوند؛ یکی مربوط به داده‌های بدون استانداردسازی و دیگری مربوط به داده‌های استانداردسازی شده. محور افقی مقادیر K و محور عمودی میانگین خطای طبقه‌بندی را نشان می‌دهد. همچنین عنوان، برچسب محورها، خطوط راهنما و راهنمای نمودار برای خوانایی بهتر اضافه شده‌اند. در نهایت نمودار در فایل `error_plot.png` ذخیره شده و روی صفحه نمایش داده می‌شود تا بتوان بهترین مقدار K و تأثیر استانداردسازی بر عملکرد مدل را به صورت بصری مشاهده کرد.



THYROID DISEASE CLASSIFICATION USING KNN

KNN WITHOUT STANDARDIZATION

K = 1 | Average Classification Error = 0.2700
 K = 2 | Average Classification Error = 0.2800
 K = 3 | Average Classification Error = 0.2700
 K = 4 | Average Classification Error = 0.2800
 K = 5 | Average Classification Error = 0.2800
 K = 6 | Average Classification Error = 0.3000
 K = 7 | Average Classification Error = 0.3200
 K = 8 | Average Classification Error = 0.2900
 K = 9 | Average Classification Error = 0.3100
 K = 10 | Average Classification Error = 0.2900

KNN WITH STANDARDIZATION

K = 1 | Average Classification Error = 0.3800
 K = 2 | Average Classification Error = 0.3600
 K = 3 | Average Classification Error = 0.3600
 K = 4 | Average Classification Error = 0.3500
 K = 5 | Average Classification Error = 0.3700
 K = 6 | Average Classification Error = 0.3200
 K = 7 | Average Classification Error = 0.3600
 K = 8 | Average Classification Error = 0.3200
 K = 9 | Average Classification Error = 0.3200
 K = 10 | Average Classification Error = 0.3100

FINAL RESULTS:

Without Standardization --> Best K = 1 | Minimum Error = 0.2700
 With Standardization --> Best K = 10 | Minimum Error = 0.3100

نتایج به دست آمده نشان می‌دهند که رفتار مدل KNN در این مسئله تا حدی غیرمنتظره است. مطابق انتظار تئوری، معمولاً استانداردسازی داده‌ها باید عملکرد KNN را بهبود دهد، زیرا این الگوریتم مستقیماً بر پایه فاصله اقلیدسی عمل می‌کند و اختلاف مقیاس بین ویژگی‌ها می‌تواند محاسبات فاصله را تحت تأثیر قرار دهد. با این حال، در این آزمایش مشاهده می‌شود که خطای طبقه‌بندی در تمام مقادیر K پس از استانداردسازی افزایش یافته است. بهترین نتیجه بدون استانداردسازی با خطای ۰.۲۷ برای $K=1$ و $K=3$ حاصل شده، در حالی که بهترین نتیجه پس از استانداردسازی خطای ۰.۳۱ برای $K=10$ است. این موضوع نشان می‌دهد که ویژگی‌هایی که دارای مقیاس بزرگ‌تر بوده‌اند احتمالاً اطلاعات تشخیصی مهمی برای تفکیک کلاس‌ها در اختیار مدل قرار می‌دهند و استانداردسازی باعث کاهش وزن نسبی این ویژگی‌ها شده است. به عبارت دیگر، مقیاس اولیه داده‌ها احتمالاً حامل اطلاعات مفیدی بوده که پس از نرمال‌سازی از بین رفته است.

با بررسی نمودار نیز می‌توان مشاهده کرد که منحنی مربوط به داده‌های بدون استانداردسازی تقریباً در تمام مقادیر K پایین‌تر از منحنی استانداردسازی قرار گرفته است. علاوه بر این، کمترین خطا در حالت بدون استانداردسازی برای K های بسیار کوچک رخ داده است.

انتخاب $K=1$ معمولاً نشانه مدلی با واریانس بالا است که به شدت به نزدیک‌ترین نمونه آموزشی وابسته است و می‌تواند مستعد Overfitting باشد. با این حال اگر Overfitting شدید وجود داشت انتظار می‌رفت که با افزایش K ، خطا به شکل محسوسی کاهش یابد؛ در حالی که در اینجا افزایش K از ۱ به ۱۰ نه تنها بهبود ایجاد نکرده بلکه خطا را به تدریج افزایش داده است. این رفتار نشان می‌دهد که ساختار داده احتمالاً به گونه‌ای است که همسایگان بسیار نزدیک اطلاعات ارزشمندتری نسبت به میانگین‌گیری روی همسایگان بیشتر ارائه می‌کنند و بنابراین مدل در K های کوچک عملکرد بهتری دارد.

در حالت استانداردسازی، روند کاملاً متفاوت است. در K های کوچک (به ویژه $K=1$) خطا به مقدار نسبتاً بالای ۰.۳۸ می‌رسد که نشان می‌دهد پس از استانداردسازی، نزدیک‌ترین همسایه‌ها دیگر نماینده‌های مناسبی برای کلاس نمونه‌ها نیستند. با افزایش K ، خطا کاهش پیدا می‌کند و بهترین عملکرد در $K=10$ مشاهده می‌شود. این الگو معمولاً نشان‌دهنده Underfitting نسبی در K های بزرگ نیست، بلکه بیانگر آن است که استانداردسازی ساختار فاصله‌ها را به نحوی تغییر داده که استفاده از یک همسایه یا تعداد کمی همسایه بسیار ناپایدار شده است و مدل تنها با میانگین‌گیری روی تعداد بیشتری از همسایگان می‌تواند بخشی از این مشکل را جبران کند. با این وجود حتی بهترین عملکرد حالت استانداردسازی نیز از بدترین مقادیر حالت بدون استانداردسازی ضعیف‌تر است که نشان می‌دهد استانداردسازی در این مجموعه داده سودمند نبوده است.

Conclusion

بر اساس نتایج اعتبارسنجی متقابل پنج‌بخشی، استفاده از استانداردسازی برای این مجموعه داده منجر به بهبود عملکرد KNN نشده و برعکس باعث افزایش خطای طبقه‌بندی شده است. بهترین نتیجه مربوط به حالت بدون استانداردسازی با خطای ۰.۲۷ و $K=1$ است، در حالی که بهترین نتیجه پس از استانداردسازی خطای ۰.۳۱ را نشان می‌دهد. این یافته حاکی از آن است که مقیاس طبیعی ویژگی‌های دیتاست تیرئید احتمالاً دارای ارزش تفکیکی بوده و استانداردسازی این اطلاعات را تضعیف کرده است. بنابراین برای این دیتاست خاص، استفاده از KNN بدون استانداردسازی عملکرد بهتری ارائه داده است؛ هرچند انتخاب $K=1$ می‌تواند نشانه حساسیت بالای مدل به داده‌های آموزشی باشد و پیشنهاد می‌شود در مطالعات بعدی با بررسی توزیع ویژگی‌ها، تحلیل اهمیت ویژگی‌ها و استفاده از روش‌های اعتبارسنجی بیشتر، علت دقیق این رفتار غیرمعمول مورد ارزیابی قرار گیرد.

برای اطمینان از صحت پیاده‌سازی الگوریتم به صورت From Scratch، از نسخه‌ی آماده‌ی scikit-learn نیز استفاده شد. این مقایسه نشان می‌دهد که هر چند مقادیر دقیق خطا و مقدار بهینه‌ی (K) در دو پیاده‌سازی کاملاً یکسان نیست (به دلیل تفاوت در نحوه‌ی نمونه‌برداری فولدها، تصادفی‌سازی، و جزئیات داخلی الگوریتم مانن tie-breaking)، اما روند کلی نتایج در هر دو حالت مشابه است. بنابراین می‌توان نتیجه گرفت که پیاده‌سازی دستی الگوریتم صحیح بوده است. همچنین مشاهده می‌شود که در هر دو روش، اعمال استانداردسازی لزوماً منجر به بهبود عملکرد نشده و در این دیتاست خاص، در برخی موارد حتی باعث افزایش خطای طبقه‌بندی شده است. این موضوع نشان‌دهنده حساسیت مدل KNN به ساختار توزیع ویژگی‌ها و ماهیت خاص داده‌ها است.

در نتایج حاصل از scikit-learn مشاهده شد که بدون اعمال استانداردسازی، کمترین خطا برابر با 0.2600 و در ($K=1$) به دست آمده است، در حالی که پس از اعمال نرمال‌سازی، کمترین خطا به 0.3100 و در ($K=3$) رسیده است. این اختلاف بیانگر آن است که در این دیتاست، مقیاس برخی ویژگی‌ها دارای اطلاعات معنادار برای تفکیک کلاس‌ها بوده و استانداردسازی باعث کاهش قدرت تمایز برخی فاصله‌ها شده است.

بررسی نتایج نشان می‌دهد که این دیتاست دارای ویژگی‌های خاصی است که باعث رفتار غیرمعمول در الگوریتم KNN شده است. نخست آن که توزیع کلاس‌ها کاملاً نامتوازن است؛ به طوری که برخی کلاس‌ها دارای تعداد نمونه بسیار کم (حتی تنها ۲ نمونه) هستند. این موضوع باعث می‌شود مدل در فرآیند یادگیری و به خصوص در اعتبارسنجی متقابل، نسبت به این کلاس‌ها عملکرد پایدار نداشته باشد و حتی در برخی fold ها این کلاس‌ها به درستی نمایندگی نشوند. این مسئله همچنین هشدار sklearn مبنی بر کم بودن تعداد نمونه در برخی کلاس‌ها نسبت به تعداد fold ها را توجیه می‌کند.

از سوی دیگر، وجود ویژگی‌هایی با مقیاس‌های بسیار متفاوت و همچنین وجود داده‌های خاص یا outlierها باعث شده است که فاصله اقلیدسی به صورت یکنواخت رفتار نکند. در نتیجه، در این مسئله خاص، استانداردسازی به جای بهبود عملکرد، موجب از بین رفتن برخی روابط طبیعی بین ویژگی‌ها شده و کاهش دقت مدل را به همراه داشته است.

در مجموع، نتایج نشان می‌دهد که:

- پیاده‌سازی From Scratch صحیح بوده است.
- نتایج با scikit-learn از نظر روند رفتاری تأییدکننده‌ی آن هستند.
- دیتاست دارای عدم توازن شدید کلاس‌ها و ساختار ویژگی‌های غیرهم‌مقیاس و حساس به فاصله است.
- به همین دلیل، KNN در این مسئله به شدت به انتخاب preprocessing به ویژه scaling حساس بوده و رفتار غیرخطی و غیرقابل پیش‌بینی در خطا نشان داده است.